APPROVED ARCHITECTURE, EXPLAINED

How the Ripples diagrams fit together

Seven views of one system: who talks to whom, where the code will live, how a write is committed, which module owns which records, and the relationships PostgreSQL must protect.

This guide explains the architecture diagrams. It does not add product scope. Protected purchase, Ripple currency, wallets, transfers, gifting, and creator payouts remain outside the launch. The API already connects to PostgreSQL. The domain tables below are the target model. The only table in the database today, notes, is a boot placeholder and is not part of this model.

1. One system, seven views

Each diagram is a zoom level, not a separate product. Start with the landscape, then the code layout, then the write path, then the modules inside the API, then the two state machines those modules must obey. The seeker path is the same system walked in product order.

Diagram	Question it answers	Where it sits
1. System context	Which people and systems exist, and which way data moves.	The whole landscape. Every other diagram happens inside one of these boxes.
2. Target repository	Where the code for those boxes will live in this repo.	Inside the web shell, the mobile shell, and the API.
3. Command and outbox	How a change becomes durable, and how work continues after the commit.	The write arrow from a shell, through the API, into PostgreSQL, then onto Redis.
4. Module ownership	Which part of the API may create or change which records.	The inside of the API box.
5. Listing lifecycle	Which listing states are legal, and which actions each state allows.	The Inventory records in PostgreSQL.
6. Protected inspection	How one held fee or deposit is allowed to end.	The Protection record and its ledger entries.
7. Seeker path	The order a person moves from browsing to a protection outcome.	A walk across diagrams 1, 3, 4, 5, and 6.

> Read rule: a shell may ask for a command and may show a projection. It does not set identity, ownership, freshness, payment, protection, or settlement. Those are committed in PostgreSQL by the API, or they do not happen.

2. System context

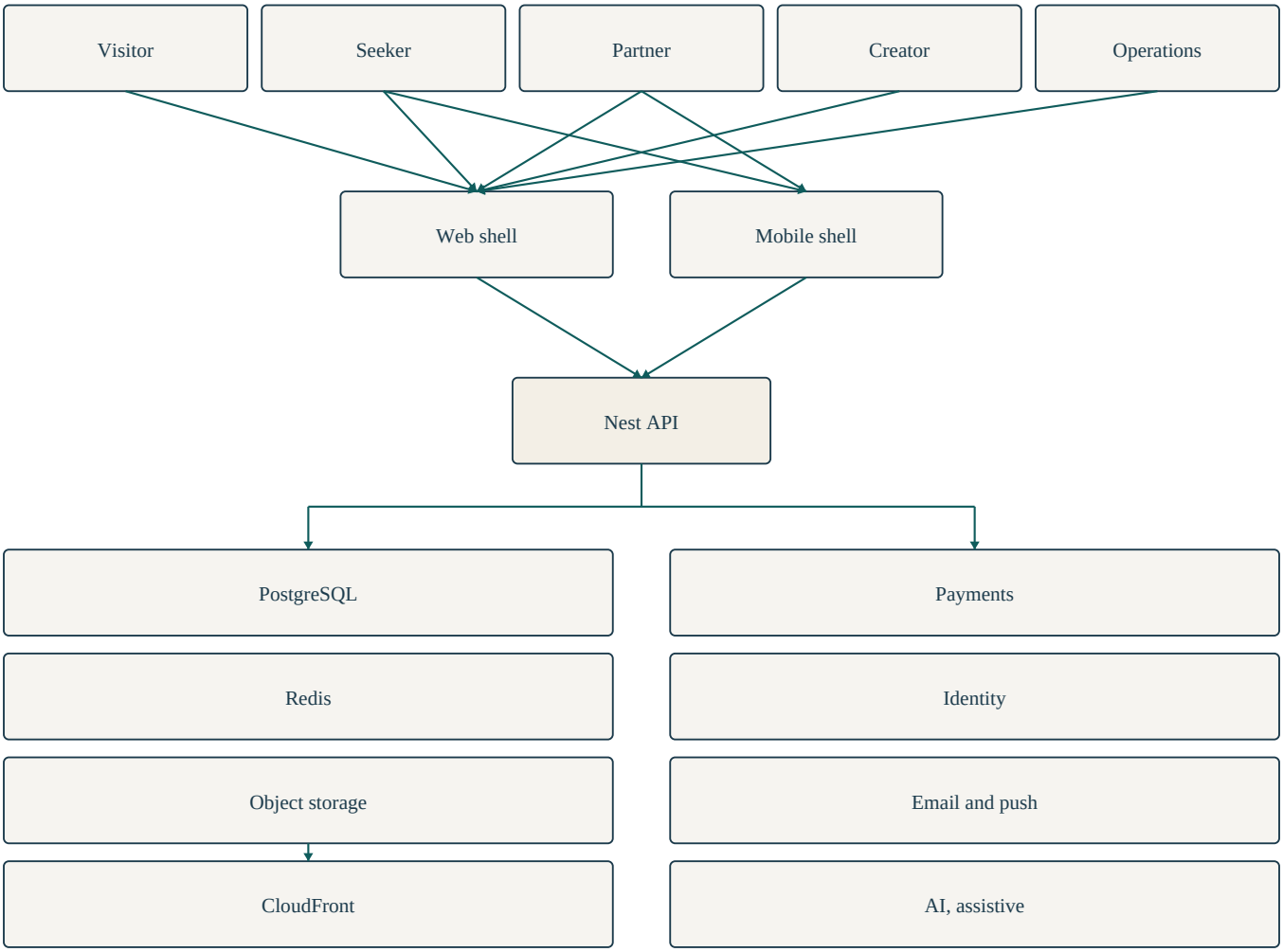


Diagram 1. System context. Seeker and partner use both shells. Visitor, creator, and operations use the web shell. The API writes the records on the left and calls the providers on the right. The AI link is assistive. Object storage feeds CloudFront.

This is the map. Visitors, seekers, partners, creators, and operations users reach Ripples through a web shell or a mobile shell. One person can act as a seeker and as a partner without a second account. The shell sends the request. The API decides who they are and what the active organization, role, and permission allow.

PostgreSQL is the system of record for canonical state, permissions, listing versions, appointments, protection records, cases, and the financial ledger. Redis holds cache, rate limits, sessions, short-lived coordination, and the BullMQ queue. Redis is never a second copy of a listing or of money. If Redis is empty, PostgreSQL can still tell the truth.

Media files go to private object storage. CloudFront delivers them. The database keeps the metadata, the owner, and the permission. Payment, identity, email, and push providers sit behind adapters. The shell never calls those providers directly. AI may help rank or draft. Discovery still runs on a deterministic ranking when AI is unavailable, and an AI suggestion does not become a listing fact until a normal command accepts it.

How the other diagrams attach to this map

- Web and mobile on this map are apps/web and apps/mobile in the repository diagram.
- The API box is apps/api. The module diagram is the floor plan of that box.
- The arrow from the API into PostgreSQL is the commit in the command diagram. The arrow onward to Redis is the queue, after the commit.

- The listing diagram and the protection diagram are state machines of records inside PostgreSQL.
- The seeker path starts in a shell on this map and ends as rows in PostgreSQL.

3. Target repository

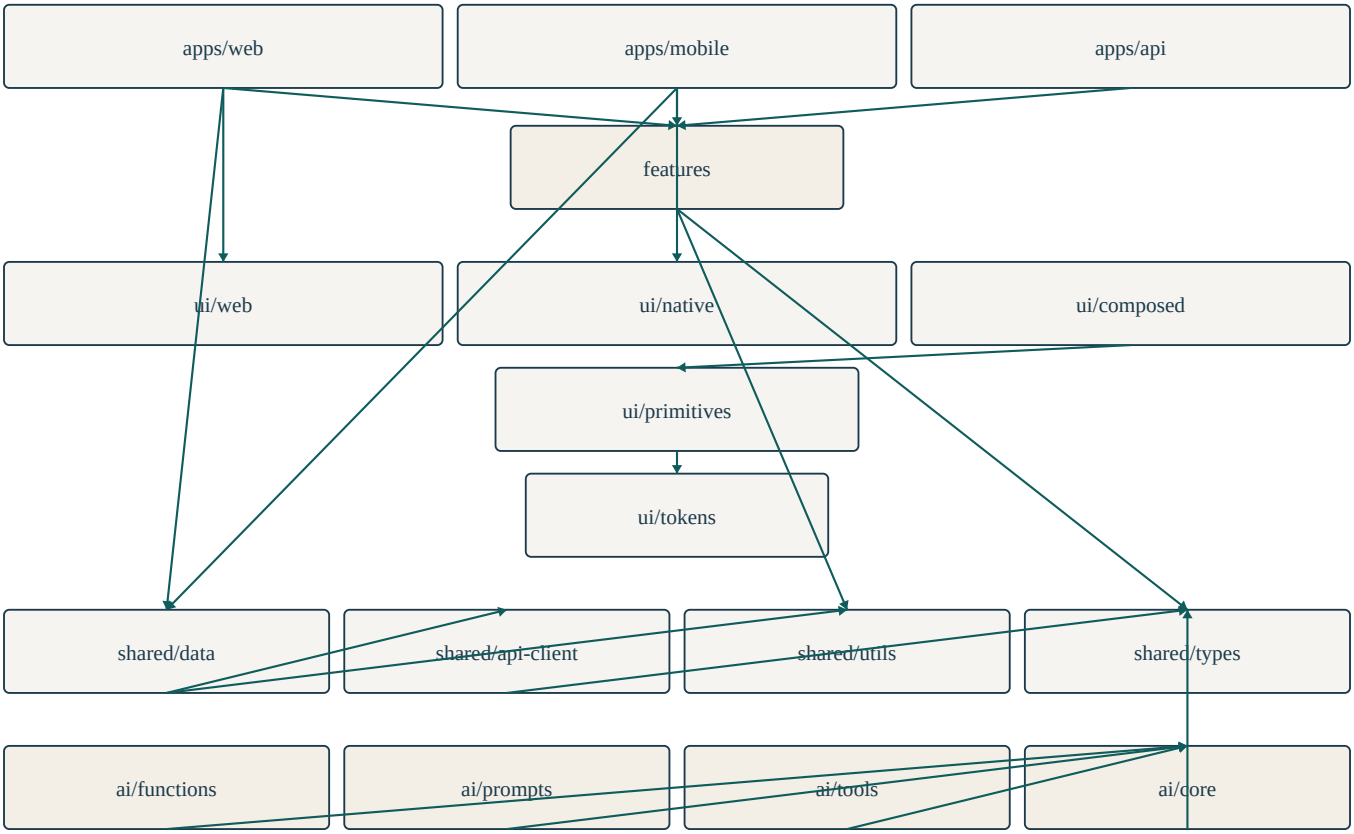


Diagram 2. Target repository.

The repository today is one NestJS application under src/. That tree is the starting point. The same repository becomes the modular monolith: apps, features, shared packages, the UI ladder, and the AI packages.

apps/api keeps composition, durable workflows, repositories, provider adapters, workers, and operational modules. apps/web and apps/mobile compose screens and routes. features own vertical product workflows such as inventory, inquiry, and protected inspection. shared/types, shared/api-client, shared/data, and shared/utils own contracts, the typed client, client-side data access, and pure logic. ui holds tokens, primitives, composed components, and the web and native layers. ai/core, ai/functions, ai/prompts, and ai/tools stay generic.

The dependency arrows are a rule, not a suggestion. Shared packages do not import apps, features, UI, or AI. AI may depend on shared packages and does not depend on UI. Applications compose features. A feature does not become a second API, and a shell does not become the owner of a business rule.

> Connection: if a rule is true in PostgreSQL, it is enforced in apps/api or in a feature that apps/api composes. A shared package or a UI package cannot be the place that decides a hold, a listing version, or a permission.

4. Command and outbox

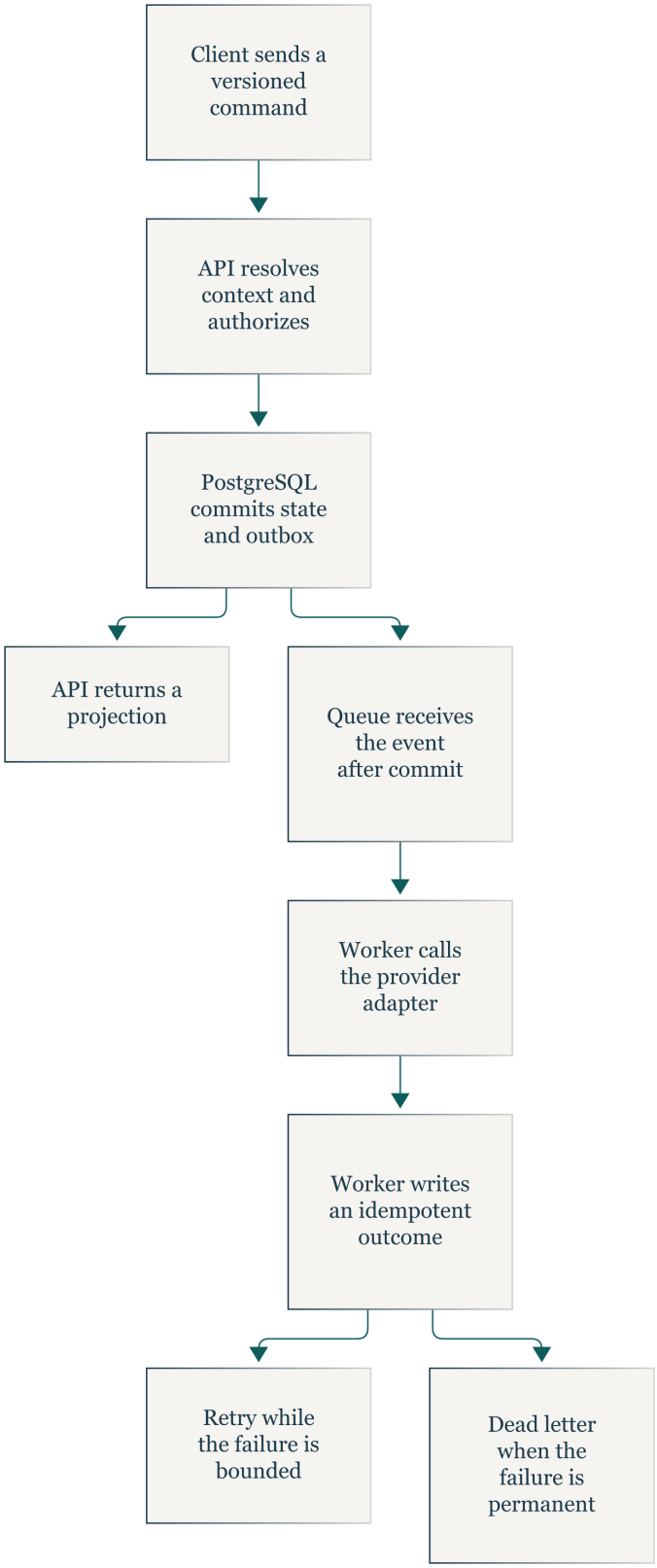


Diagram 3. Command and outbox.

This is the only write path. The client sends a versioned command under /api/v1. The API resolves the session and the active context, checks permission, and validates the payload. The business change and an outbox row are inserted in one PostgreSQL transaction. The client then receives a projection or an acceptance. Nothing is published before that commit succeeds.

After the commit, a publisher reads the outbox and places the event on the queue. A worker may call a provider adapter or write a follow-on record. Workers are idempotent, because the same event can be delivered twice. Retryable failures return to the queue with bounded backoff. A permanent failure goes to a dead-letter path that an operator can see.

Protected actions and payment actions fail closed. If PostgreSQL cannot confirm the current state, or the payment provider cannot confirm the hold, the API does not mark funds released and does not invent a successful appointment.

What this diagram borrows from the others

- The client is a shell from the system map. The active context comes from the identity module.
- The module that performs the command is one box in the ownership diagram. It may write only its own records.
- A listing command must land on a legal state in the listing diagram. A protection command must land on a legal outcome in the protection diagram.
- The outbox event names the same subject that the seeker path just acted on: a listing version, an inquiry, an appointment, or a hold.

5. Module ownership

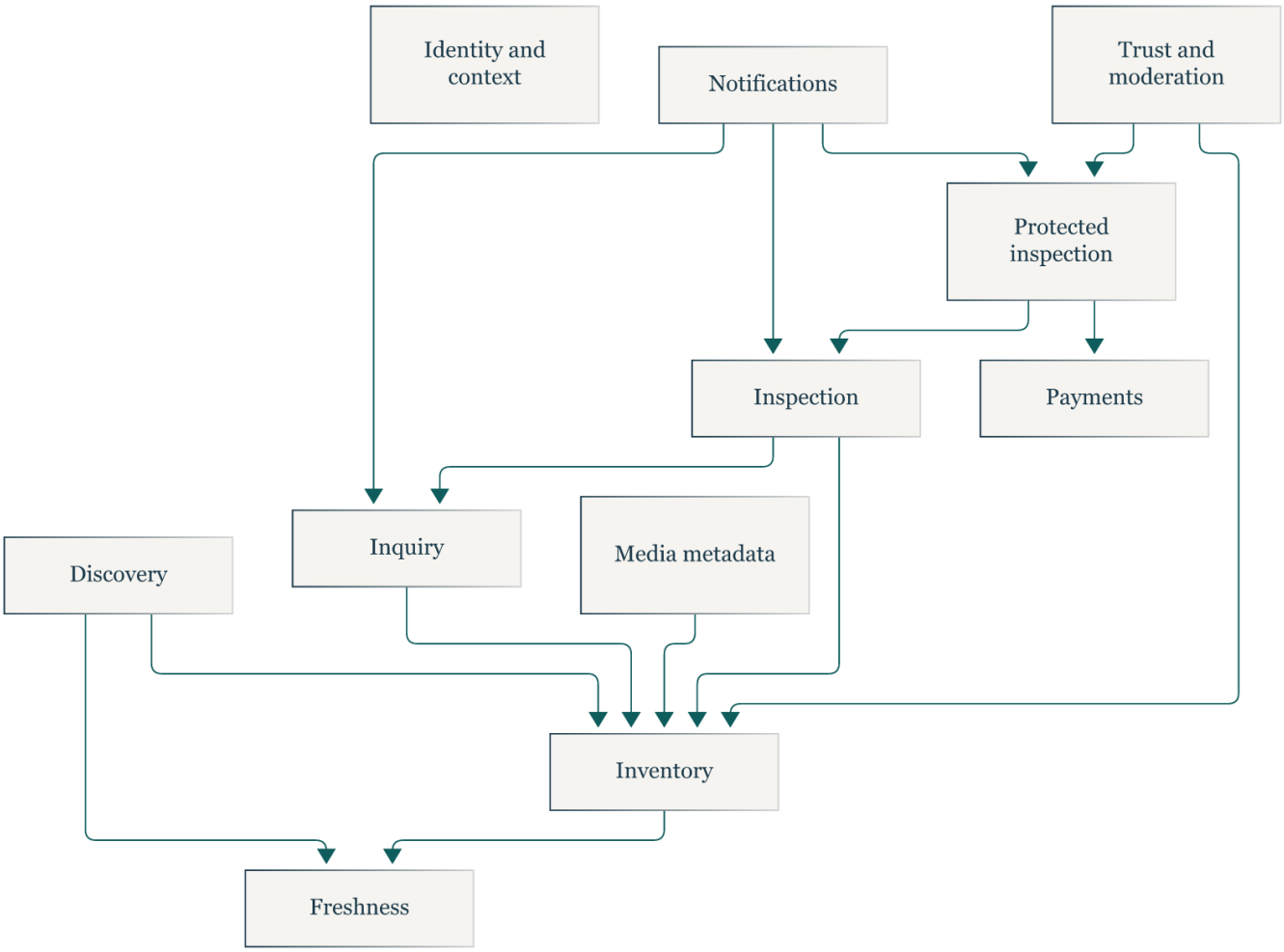


Diagram 4. Module ownership. An arrow means the source depends on the target.

Inside the API, ownership is vertical. Identity and context own the person, the organization membership, and the active role. Inventory owns property, project, unit, listing, listing version, and freshness. Media owns metadata for files that live in object storage. Discovery reads current, policy-eligible inventory and does not invent commercial terms. Inquiry owns the seeker's question against a listing version. Inspection owns the appointment and its outcome. Protected inspection owns eligibility and the hold. Payments owns provider calls and the ledger. Trust owns review, moderation, and case resolution. Notifications own delivery requests.

The arrows between modules are the allowed collaborations. Inventory owns freshness. Media and discovery read inventory. Inquiry reads the current listing version. Inspection follows an inquiry and keeps the listing version that was shown at booking. Protected inspection depends on inspection and on payments, and on nothing else. It does not read the feed to decide a refund, and creator attribution does not grant settlement authority. Trust may act on inventory and on protection. Notifications follow inquiry, inspection, and protection after those modules have committed.

> Connection: a module arrow is also a database relationship. Inspection can store the id of a listing version because it is allowed to depend on inventory. Protection can store the id of an appointment and the id of a ledger entry. Protection does not store a content id or a creator id as the reason a hold was released.

6. Listing lifecycle

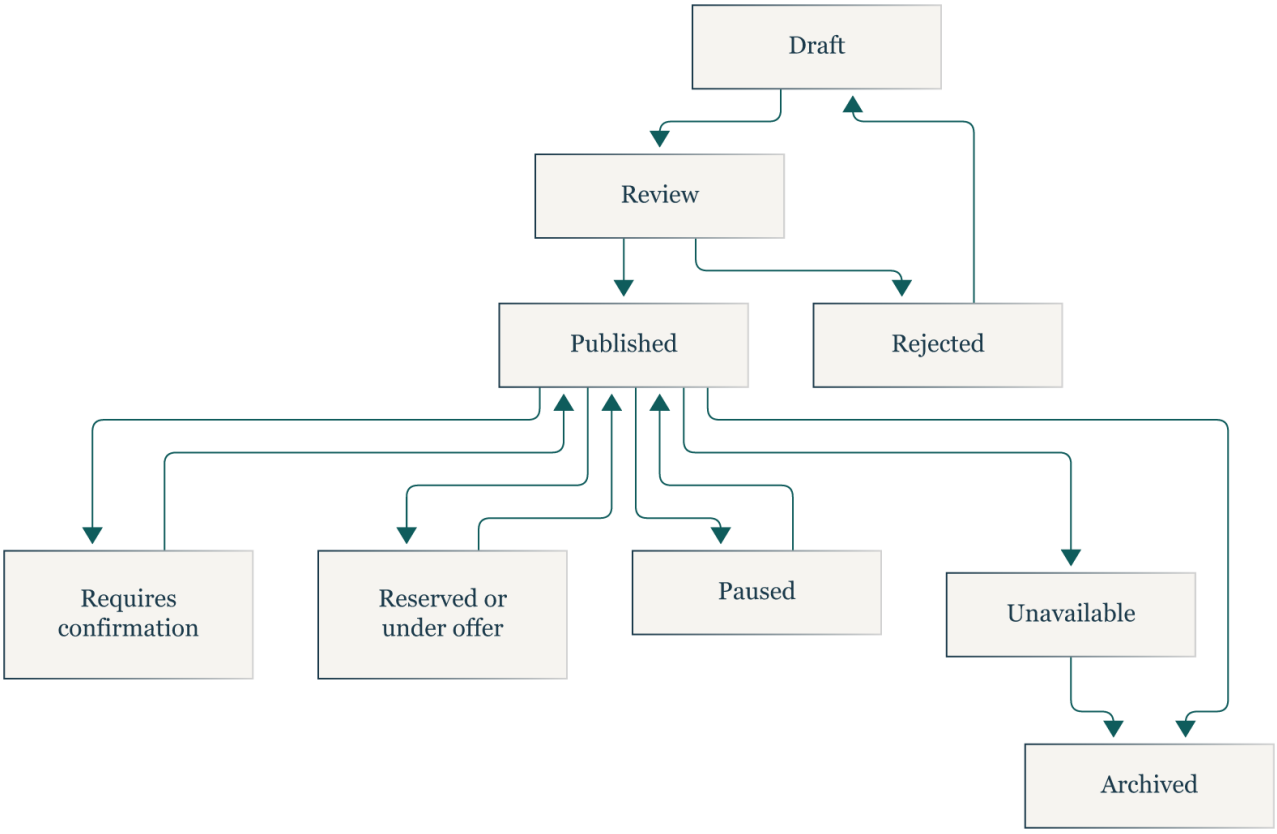


Diagram 5. Listing lifecycle.

A listing is the current market offer for one property or one unit. Its states are Draft, Review, Published, Requires Confirmation, Reserved or under offer, Unavailable, Paused, Rejected, and Archived. A partner edits a draft. Submit moves it to Review. Published is reached only after validation, authorization, duplicate checks, moderation, and an initial freshness confirmation. Choosing Publish does not by itself make the listing public. Low-risk inventory may pass those checks without a manual wait. Rejected returns to Draft for correction.

From Published, the listing can require confirmation, become reserved, pause, become unavailable, or be archived. Requires Confirmation, Reserved, and Paused can return to Published when the checks pass again. Unavailable can be archived.

Requires Confirmation still allows a non-binding inspection request. It does not allow a protected fee, and it does not allow the appointment to be confirmed. Stale, withdrawn, and materially changed listings stay in history. They are not shown as current, and they are not accepted for a new protected action.

7. Protected Single Inspection

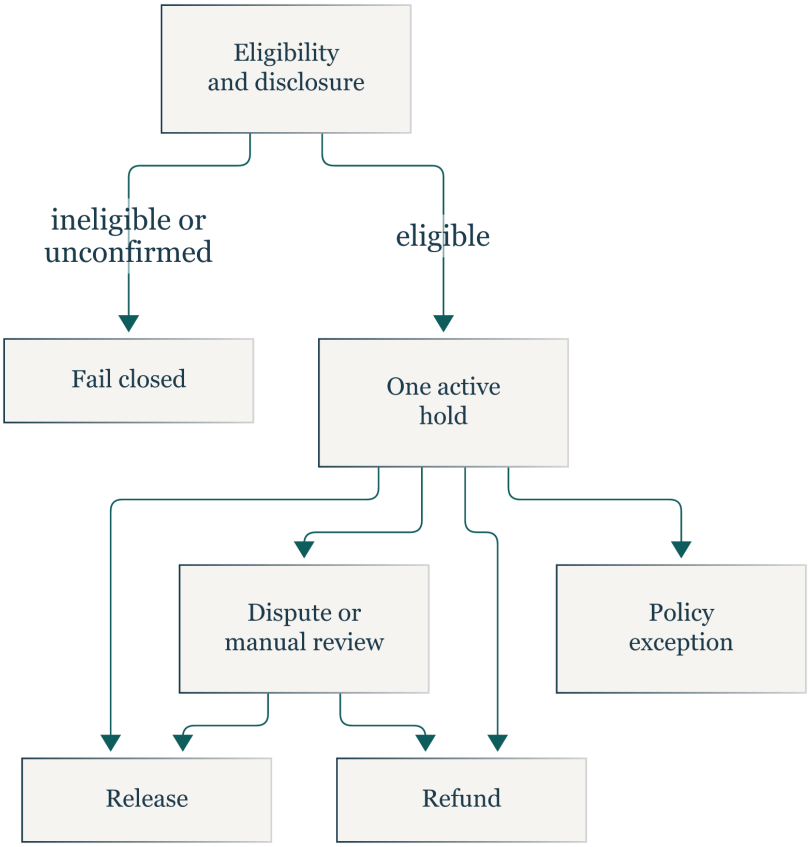


Diagram 6. Protected Single Inspection.

Protection is a narrow hold around one inspection appointment. It is not a general escrow, and it does not transfer title or replace legal due diligence. Before any charge, the API checks the listing, the partner, the market, the payment method, and freshness. Failure ends the command. No hold row is created.

When the check passes, that appointment has one active hold. The hold ends in exactly one of these outcomes: release, refund, dispute or manual review, or a policy exception. A dispute can later release or refund. It is not resolved by silence or by the appointment merely existing. Funds move only when the milestone and the evidence required by the policy are recorded. The amount is fiat.

The seeker, the partner, and authorized operations can see the payment state that applies to them. Settlement commands are idempotent, so a retry does not release or refund twice. Every movement is an append-only ledger entry.

> Connection to the listing diagram: the eligibility check reads the listing state and the latest freshness confirmation. Published and fresh can proceed. Requires Confirmation, stale, paused, unavailable, and archived fail closed. Connection to the command diagram: the hold, the first ledger entry, and the outbox event commit together.

8. Seeker path

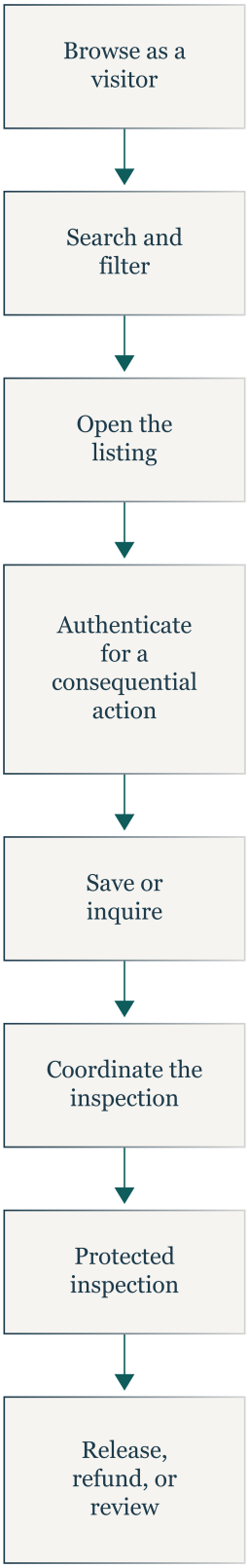


Diagram 7. Seeker path.

The seeker path is the product order that crosses the other diagrams. A visitor can browse, search or filter, and open a listing with no session. The feed and the detail page are reads. They show the current listing version, the accountable partner, freshness, availability, and whether protection is offered. Sign-in does not block that reading.

A consequential action requires authentication: save, inquire, coordinate an inspection, book protection, and receive a release, refund, or manual review. Each of those actions is a command. The inquiry stores the seeker, the listing

version, the partner, and the time. The appointment keeps the listing version and the material terms shown at booking. A later non-material edit does not rewrite that appointment. A material change to price, availability, fees, the property or unit, location, access, or protection requires re-confirmation or a policy outcome such as reschedule, cancel, or refund.

9. One booking, walked across the diagrams

A seeker books a protected inspection on a listing they opened as a visitor. The steps below are one action, named in the diagram that governs each step.

Step	Diagram	What is true at that step
Open the listing	Seeker path and system context	The shell reads through the API. Discovery returns the current listing version. No session is required.
Authenticate	Seeker path and module ownership	Identity resolves the person and the active context. The client does not choose a permission.
Check the listing	Listing lifecycle	The listing is Published and the freshness confirmation is inside policy. Otherwise the fee is refused.
Create the appointment	Module ownership	Inspection stores the listing version shown at booking, the seeker, and the partner.
Take the hold	Protected inspection	Eligibility has passed. This appointment has one active hold. Payments is the only money collaborator.
Commit	Command and outbox	Appointment, hold, ledger entry, and outbox event commit in one PostgreSQL transaction, or the command fails.
Continue after commit	System context	Redis carries the queue. A worker calls the payment adapter and sends notifications. PostgreSQL remains the record.
Where the code lives	Target repository	The feature is composed by apps/api. The shell sends the command and renders the projection.

10. Relationships PostgreSQL stores

PostgreSQL is the relational store for these records. The diagrams draw it as one box because every canonical relationship lives there together, in one transactional database. Redis, object storage, and a provider's own dashboard are not extra places where these relationships are authored. A relationship below is a foreign key the owning module is allowed to write. A listing has many versions, and exactly one of those versions is marked current.

Identity and context

From	Relationship	To
Person	has many	Membership
Organization	has many	Membership
Membership	belongs to one	Person
Membership	belongs to one context	Organization or personal context
Session	belongs to one	Person
Session	points at one active	Membership

Personal activity and organization activity stay separated by membership. Switching context changes the active membership before a consequential command. Operations memberships are least-privilege and are themselves audited.

Inventory, freshness, media, and content

From	Relationship	To
Project	may group many	Property
Property	may belong to one	Project
Property	has many	Unit
Unit	belongs to one	Property
Listing	offers exactly one subject	Property or Unit
Listing	has one accountable	Membership (partner)
Listing	has many, one current	Listing version
Listing version	belongs to one	Listing
Listing	has many	Freshness confirmation
Freshness confirmation	records one actor	Membership
Media metadata	belongs to an owner	Membership
Media metadata	may attach to one subject	Property, unit, or listing
Content	references one canonical subject	Property, project, unit, or listing version

A listing subject is a property or a unit, not both. The current listing version is the only version discovery may present as the offer. Older versions remain for audit, for the inquiry that cited them, and for the appointment that was booked against them. Content points at that canonical subject. It does not have its own price, availability, or protection flag. The media row stores the storage key and the permission. The bytes stay in object storage.

Inquiry, inspection, protection, and money

From	Relationship	To
Inquiry	is from one	Membership (seeker)
Inquiry	cites one	Listing version
Inquiry	is addressed to one	Membership (partner)
Appointment	books one	Listing version
Appointment	has seeker and partner	Membership
Appointment	has many	Outcome
Protection record	covers one	Appointment
Appointment	has at most one active	Protection record
Protection record	has many append-only	Ledger entry
Ledger entry	belongs to one	Protection record
Audit record	references the changed subject	Listing, appointment, protection, or membership
Outbox event	is written in the same commit	The change it describes

The appointment stores the listing version presented at booking. A later version of the listing does not replace that link. The protection row exists only after eligibility passes, and only one active hold may point at that appointment. Ledger rows are added, not edited. A correction is a new ledger row. The audit row and the outbox row are written in the transaction that made the change, so a committed fact and the news of that fact cannot diverge.

11. Invariants these relationships protect

- Exactly one listing version is current.
- Every consequential command has a real person, a real membership, and a permission for that action.

- A protected appointment has at most one active hold.
- Payment and settlement commands are idempotent. A retry does not create a second hold, release, or refund.
- Financial records are append-only.
- Material transitions are auditable. A correction keeps the prior state and the reason.
- Content and creator attribution do not override verification, freshness, protection, or settlement.
- A stale listing version remains historical. It is not presented as current and it cannot start a new protected hold.

12. What is outside these tables

- Media bytes live in private object storage. CloudFront delivers them. PostgreSQL stores the key, the owner, the processing state, and who may see the file.
- Redis stores cache, rate limits, sessions, and queue messages. Those entries expire or can be rebuilt. They are not the listing, the appointment, or the ledger.
- Provider systems store their own charge identifiers. Ripples stores the correlation on the ledger and reconciles it. A provider callback is untrusted until it is authenticated and matched to a Ripples hold.
- AI output stays outside canonical tables until a validated command writes a normal inventory, inquiry, or moderation record.

13. Where the API is today

The running API is a NestJS service. It opens PostgreSQL with POSTGRES_HOST, POSTGRES_PORT, POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB, and POSTGRES_SSL. Schema changes currently apply through TypeORM synchronize, and the seeded table is notes. That is a temporary boot path.

The target, already decided, is Prisma migrations over the same PostgreSQL settings, a versioned /api/v1 prefix, and the relationships in this guide. The notes table does not survive that end state. Until those migrations exist, this document is the relationship map. It is not a claim that the tables have already been created.